

Trail: A Deterministic-First Log Template Miner with a Per-Decision Audit Trail

Eduard Beloiu
MbitAI

September 2026

Abstract

Log template miners usually return templates and aggregate scores, but not the sequence of clustering decisions that produced them. Trail is a deterministic, single-pass miner that writes one audit record per input line: cluster, similarity, decision, and resulting template. An optional local 2B-parameter model reviews low-confidence joins without changing the deterministic parse. On three LogHub-2k datasets, one global Trail configuration ties tuned Drain on Linux grouping accuracy (GA 0.690), but trails it on OpenSSH (0.542 vs. 0.789); on Apache its GA is 0.984 vs. 1.000. Ablations show that anchors help on Linux and OpenSSH but fragment Apache, while similarity thresholds from 0.4 to 0.6 do not change these runs. On the synthetic SecOps-2k set, Trail improves GA from 0.772 to 0.967 and grouping F1 from 0.295 to 0.863 over Drain. A case study shows how line-level records distinguish a defensible disagreement with ground truth from a low-confidence merge error. The evaluation uses pinned configurations, dataset hashes, and reproducible result artifacts.

1 Introduction

Template mining turns raw log lines into event templates by grouping lines that differ only in variable parts. Standard evaluations compare the final groups and templates with labeled data [1, 2]. These outputs do not identify the earlier decision that caused a bad merge. When a parser joins two security events that should stay apart, an aggregate score measures the effect but does not record when the join happened or why the parser accepted it.

Trail records each clustering decision in one JSON record per input line. Human review and model assist cite those records instead of re-parsing, preserving a direct path from each final template to the decisions that formed it. This report reviews related work (§2), then describes the miner (§3), the optional local-model review loop (§4), a worked case study of two errors hidden by one metric (§5), a three-parser evaluation on LogHub-2k plus an ablation (§6), known limits (§7), and reproduction instructions (§8).

This technical report makes three contributions:

- a deterministic miner that records the evidence for every clustering decision;
- a separate local-model review path whose proposals cannot alter the base parse without passing explicit gates; and
- an evaluation with a per-decision OpenSSH case study and configuration ablations that expose both gains and failure modes.

2 Related work

Spell and Drain are streaming, syntax-based parsers that cluster log tokens without model inference [4, 3]. The logparser toolkit standardized comparisons across multiple datasets and parsers [5]; LogHub-2.0 later expanded the data and introduced the GA, PA, FGA, and FTA measures used here [1].

Recent systems use language models to extract or refine templates. EFParser combines small language models with exemplar selection, caches, and template correction [6]. DeepParse uses a model once to synthesize regex masks, then applies them through a deterministic Drain runtime [7]. Matryoshka instead parses security logs into semantic schemas for downstream analysis [8]. Trail does not attempt to match the accuracy scope of these systems. It focuses on a narrower operational property: preserving each clustering decision, and keeping optional model review separate from the deterministic output.

3 Design

The miner is one pass, no models, no training. Each line joins the best matching cluster or starts a new one.

Preprocessing. The syslog header is stripped with one regex; only the message part is mined. On LogHub-2k files the harness splits the header with the same per-dataset format string used for the Drain baseline, so all parsers see identical input. Messages are split on whitespace. A small pinned list of whole-token masks replaces matching tokens with field-named wildcards (e.g. `SRC=<*>`), matching ground-truth convention.

Candidates. Only clusters whose token counts differ by at most `length_slack` (1) and whose first `anchor_tokens` (2) match exactly can merge. A candidate is further rejected when template and line disagree on an `identity_keys` field: the same `key=` prefix with different values (`PROTO=TCP` vs. `PROTO=UDP`). Identity keys name closed-vocabulary fields that separate templates; everything else may generalize through key-aware merge.

Join. Among candidates, the highest token similarity wins. Exact tokens and any template position holding `<*>` count as matches. For a template T with m tokens and a line L with n tokens, Trail computes

$$s(T, L) = \frac{1}{\max(m, n)} \sum_{i=1}^{\min(m, n)} \mathbf{1}[T_i = L_i \vee T_i = \langle * \rangle].$$

The best candidate joins when $s(T, L) \geq \text{st}$; the global configuration in this report sets `st` to 0.5. Otherwise Trail opens a new cluster. On join, differing positions generalize: tokens sharing a `key=` prefix become `key=<*>`, anything else becomes bare `<*>`, permanently. Templates only ever generalize, never split.

Why anchors. Similarity alone merges lines that differ in exactly one outcome word. Lines beginning `Failed password` and `Accepted password` agree on every later token, so a pure threshold puts them in the same template. Requiring the first two tokens to match keeps outcomes apart

without per-dataset tuning. It is a heuristic, not a principle, and the audit log shows everywhere it fires, including where it fires wrongly (§6).

Audit schema (v1). One record per line:

```
{"line": 12, "cluster": "T3", "decision": "matched",
  "similarity": 0.875, "template": "Failed password for <*> ..."}

```

`decision` is `new_cluster` (similarity 0.0) or `matched`. Readers must ignore unknown fields; writers must not rename existing ones. Model proposals and accept/reject decisions live in a separate append-only review log that cites audit line numbers, so the parse audit stays an immutable receipt.

4 Local-model assist

The deterministic core stays the parser of record. A local small model (Qwen3.8-2B distill, Q6_K, `empero-ai/Qwen3.8-2B-Distill-GGUF` [9], Apache-2.0) reviews candidates selected from the audit log: low-confidence joins (similarity under 0.7) and near-duplicate cluster pairs. Each candidate includes three cited example lines. The prompt (`trail-lm-v2`) defines SAME vs. TWO, adds domain rules (Accepted vs. Failed and BLOCK vs. ALLOW are TWO; IP/port/user-only diffs are SAME; unsure defaults to SAME), and asks one question per candidate. Anything that is not one unambiguous SAME or TWO is rejected. Accepted changes land in a separate assisted CSV; the deterministic CSV and audit never change. Accepts touching more than 10 lines are held as `needs-human` and never materialized.

The first prompt version (`trail-lm-v1`, bare SAME/TWO question) degraded SecOps-2k tight GA from 0.967 to 0.682 through two false accepts. The v2 preamble plus the hold rule contain exactly that failure mode: v2 makes zero auto-applies and holds three cases for human review, so the assisted CSV equals the deterministic parse. We include the v1 result because it motivates the v2 gate.

Measured cost on an Apple M2 with 8 GB RAM (CPU, llama.cpp server [10]): model load about 40 s at 3.7 GB resident; 16 SecOps-2k candidates in 463 s (about 29 s each); verdicts 13 reject, 3 needs-human, 0 auto-apply. The local run sent no log data to a remote service. This is one measurement of one model and prompt on one machine, not a performance benchmark.

5 Case study: two merges, one number

On OpenSSH_2k, Trail trails Drain in grouping accuracy (GA 0.542 vs. 0.789). Nearly all of the gap sits in two parsed clusters holding 908 of 2000 lines. GA compresses them into one number; the audit records show they are qualitatively different events.

Exhibit A: confident merge. Cluster T5 (494 lines) merges two truth templates that differ by a trailing optional `user=` field. Listing 1 shows the birth record (line 5) and the merge record (line 28, similarity 0.90); later joins of the same variant come in at 1.0. The merge is stable, not borderline. A human with this receipt can reasonably side *with the miner* against ground truth: same event, optional field. GA counts all 110 lines wrong either way. Scores cannot express “wrong by the metric, right by the event”; the receipt preserves the disagreement in a form a reviewer can check.

Exhibit B: borderline swallowing. Cluster T8 (414 lines) absorbs a single `disconnected` by `user` line into 413 `Bye Bye [preauth]` lines at line 964, similarity 0.625, some 950 lines after the cluster was born. GA barely changes for one line in 2000, while template-level FGA counts the failed template. Because the similarity is below 0.7, the LM-assist loop in §4 selects this record for review. The same evidence explains the parse and supplies the review queue.

Together, the records show why per-decision evidence matters. The first merge may reflect a ground-truth boundary that is too strict; the second is a low-confidence error routed to review by its own record.

Listing 1: The four audit records behind both merges (line numbers and similarities verbatim).

```
{
  "line": 5, "cluster": "T5", "decision": "new_cluster",
  "similarity": 0.0, "template": "pam_unix(sshd:auth):
  authentication failure; logname= uid=0 euid=0 tty=ssh
  ruser= rhost=173.234.31.186 "}
{
  "line": 28, "cluster": "T5", "decision": "matched",
  "similarity": 0.9, "template": "pam_unix(sshd:auth):
  authentication failure; logname= uid=0 euid=0 tty=ssh
  ruser= rhost=<*> <*>"}
{
  "line": 14, "cluster": "T8", "decision": "new_cluster",
  "similarity": 0.0, "template": "Received disconnect from
  52.80.34.196: 11: Bye Bye [preauth]"}
{
  "line": 964, "cluster": "T8", "decision": "matched",
  "similarity": 0.625, "template": "Received disconnect
  from <*> 11: <*> <*> <*>"}
}
```

6 Evaluation

Scorer for all runs: an independent Apache-2.0 implementation of the LogHub-2.0 metrics [1]. Grouping accuracy (GA) is the fraction of messages whose predicted group exactly matches their ground-truth group. Parsing accuracy (PA) is the fraction whose predicted template tokens exactly match the ground truth. FGA and FTA are template-level F1 measures for grouping and template accuracy. Data: LogHub-2.0 `2k_dataset` at pinned commit `ac4aad2` [11], plus the synthetic SecOps-2k set (2000 `sshd`, `sudo`, `firewall`, and `auditd` lines; 25 tight templates). Each result is one deterministic run on a fixed 2000-line input, so variance estimates and significance tests do not apply.

6.1 Tier A: three parsers, three datasets

Drain uses the logpai benchmark settings [3], which vary by dataset: `st` 0.5/0.39/0.6 and depth 4/6/5. Spell runs with an untuned $\tau = 0.5$ [4] and the same file formats. Trail runs one global configuration everywhere: `st`=0.5, two anchors, and slack 1. The same Trail configuration produces the SecOps results.

Table 1: LogHub-2k results. N_p is the number of predicted templates.

Dataset	Parser	GA	PA	FGA	FTA	N_p
Apache	Drain	1.000	0.694	1.000	0.500	6
Apache	Spell	0.576	0.270	0.727	0.182	5
Apache	Trail	0.984	0.694	0.233	0.140	37
Linux	Drain	0.690	0.184	0.930	0.435	112
Linux	Spell	0.598	0.088	0.736	0.151	94
Linux	Trail	0.690	0.171	0.918	0.395	115
OpenSSH	Drain	0.789	0.508	0.880	0.440	24
OpenSSH	Spell	0.252	0.121	0.488	0.195	15
OpenSSH	Trail	0.542	0.508	0.784	0.431	25

Trail ties Drain’s GA on Linux with one global configuration, although its PA and template-level scores remain lower. Spell at the default τ trails both parsers on all three datasets; tuning Spell could change that comparison. On Apache, Spell merges the `jk2_init` Found/Can’t-find pair that Trail’s anchors keep apart. Trail’s Apache and OpenSSH differences are grouping errors rather than token errors: PA ties Drain on both datasets.

Table 2: SecOps-2k tight-group results (25 ground-truth templates).

Parser	GA	PA	FGA	FTA	N_p
Drain	0.772	0.695	0.295	0.253	70
Trail	0.967	0.967	0.863	0.863	26

Table 2 is a controlled synthetic comparison, not evidence of production security-log performance. The loose 10-group variant is available in the artifact results; FTA is unchanged by construction because the loose labels alter grouping, not template strings.

6.2 Ablation: anchor sensitivity

Nine configurations were run on each dataset with the same preprocessing and scorer. The runner changes only the in-memory Trail configuration, and the **base** run reproduces the three Trail goldens. These ablations are preserved as a runner and raw JSON, but they are not CI goldens. Table 3 reports GA/FGA with predicted-template counts in parentheses.

Table 3: Trail ablation. **st** variants are identical to base on all three sets and collapsed to one row.

Variant	Apache	Linux	OpenSSH
base	0.984 / 0.233 (37)	0.690 / 0.918 (115)	0.542 / 0.784 (25)
no-anchors	1.000 / 1.000 (6)	0.186 / 0.457 (44)	0.279 / 0.622 (19)
anchors-1	1.000 / 1.000 (6)	0.684 / 0.917 (111)	0.472 / 0.723 (21)
st-0.4/0.6	identical to base		
slack-0	0.984 / 0.233 (37)	0.690 / 0.915 (116)	0.790 / 0.868 (27)
no-identity	identical to base		
no-masks	identical to base (GA/FGA/ N_p)		

Anchors have the largest effect, with opposite signs: they are essential on Linux (GA 0.690 to 0.186 without) and OpenSSH, but they cause the Apache over-fragmentation (6 to 37 templates).

Anchorless Trail and Drain produce equivalent message groupings on Apache_2k, even though their event identifiers differ. The Apache grouping gap therefore comes from the anchor rule in this configuration. One anchor keeps nearly all of the Linux gain; the second helps OpenSSH. Similarity thresholds from 0.4 to 0.6 produce identical outputs: the structural filters decide the merges, and the threshold never binds on these three inputs. Setting slack to 0 forbids both OpenSSH merges from §5 (they differ in length by 1) and reaches nearly the same GA as Drain (GA 0.790 vs. 0.789), but the two parsers still produce different partitions. Removing masks leaves grouping unchanged, but lowers OpenSSH PA from 0.508 to 0.466 and FTA from 0.431 to 0.392. Identity keys have no effect on these LogHub groups. Choosing `no-anchors` for Apache and `slack-0` for OpenSSH after seeing the labels would close their GA gaps, but would constitute per-dataset tuning; we do not report a tuned Trail aggregate.

7 Limitations

Header splitting is syntactic; timestamps, pids, and hostnames are dropped, not parsed. Masks are a small pinned whole-token list, not a general field parser. The miner is order-dependent: a different line order can give different clusters. Inputs are pinned for reproducibility, but order sensitivity was not measured. Long lines with many variable tokens can fall below threshold and fragment; the audit log records the resulting decisions.

SecOps-2k was developed alongside Trail. Its templates exercise identity keys and masks, so it is a sanity check rather than an independent benchmark. The data has no organic noise, clock skew, or interleaving and does not support claims about detector training or production security logs. The assist results cover one model, prompt, and machine. LogHub-2k ground-truth labels are disputed: SLGParser reports significant inaccuracies in the manual labels [12], and corrected releases exist [13]. This report scores against the pinned originals without relabeling. All evaluations use 2k-line sets.

8 Reproduction

Three repositories form the pipeline. LogParser-Harness evaluates Drain, Spell, and Trail on LogHub-2k and SecOps-2k [14]. LogParser-Dataset contains the synthetic SecOps-2k set and Drain baseline [15]. LogParser-Trail contains the miner, audit format, and model assist [16]. The evaluated pipeline uses Python 3.12+, uv lockfiles, dataset hashes checked on fetch, and golden JSON for every row in Table 1. The SecOps model outcomes are stored in `lm_scores.json`; the ablation and case-study records are regenerated by their companion scripts. The case-study script reads the miner’s decision state; a committed OpenSSH `audit.jsonl` is not part of the current artifact.

Table 4: Pinned artifacts (2026-09-05). Concept DOIs cover all versions.

Artifact	Commit	Tag	Version DOI
Harness [14]	55d491a	v0.2.1	10.5281/zenodo.22341500
Dataset [15]	7ab408d	v0.1.2	10.5281/zenodo.22341506
Trail [16]	a17e1de	v0.2.1	10.5281/zenodo.22341504
Concept DOIs: 10.5281/zenodo.22341499 , 10.5281/zenodo.22341505 , 10.5281/zenodo.22341503			

References

- [1] Z. Jiang et al., “A large-scale evaluation for log parsing techniques: How far are we?” ISSTA, 2024. <https://arxiv.org/abs/2308.10828>
- [2] J. Zhu et al., “LogHub: A large collection of system log datasets for AI-driven log analytics.” ISSRE, 2023. <https://arxiv.org/abs/2008.06448>
- [3] P. He et al., “Drain: An online log parsing approach with fixed depth tree.” ICWS, 2017. http://jiemingzhu.github.io/pub/pjhe_icws2017.pdf
- [4] M. Du and F. Li, “Spell: Streaming parsing of system event logs.” ICDM, 2016. <https://www.cs.utah.edu/~lifeifei/papers/spell.pdf>
- [5] J. Zhu et al., “Tools and benchmarks for automated log parsing.” ICSE, 2019. <https://arxiv.org/abs/1811.03509>
- [6] M. Wang and Y. Huo, “Small is beautiful: A practical and efficient log parsing framework.” FSE, 2026. <https://doi.org/10.1145/3797145>
- [7] A. Shetaia and S. Kauffman, “DeepParse: Hybrid log parsing with LLM-synthesized regex masks.” EASE, 2026. <https://arxiv.org/abs/2604.20553>
- [8] J. Piet et al., “Semantic-aware parsing for security logs.” 2025. <https://arxiv.org/abs/2506.17512>
- [9] empero-ai, “Qwen3.8-2B-Distill-GGUF,” Hugging Face (Apache-2.0; weights used for §4). <https://huggingface.co/empero-ai/Qwen3.8-2B-Distill-GGUF>
- [10] G. Gerganov et al., “llama.cpp,” GitHub (MIT). <https://github.com/ggerganov/llama.cpp>
- [11] LogPAI, “Loghub-2.0,” GitHub, commit `ac4aad2`. <https://github.com/logpai/Loghub-2.0>
- [12] Y. Hu, C. Wang, L. Zhao, and A. Yu, “SLGParser: Practical and efficient label-free log parsing using large language models.” ICDE, 2026. <https://icde2026.github.io/accepted-papers.html>
- [13] “Corrected versions of LogHub, LogHub 2.0, LoFi, and Hybrid log parsing datasets,” Zenodo, 2026. <https://zenodo.org/records/20752471>
- [14] MbitAI, “LogParser-Harness,” v0.2.1 (Apache-2.0). <https://github.com/TMFNK/LogParser-Harness>. doi:10.5281/zenodo.22341500
- [15] MbitAI, “LogParser-Dataset” (SecOps-2k), v0.1.2 (Apache-2.0). <https://github.com/TMFNK/LogParser-Dataset>. doi:10.5281/zenodo.22341506
- [16] MbitAI, “LogParser-Trail,” v0.2.1 (Apache-2.0). <https://github.com/TMFNK/LogParser-Trail>. doi:10.5281/zenodo.22341504